Minicurso: Kubernetes

Orquestração de Aplicações em Contêineres

Prof. Alexandre Huff

alexandrehuff@utfpr.edu.br

Universidade Tecnológica Federal do Paraná (UTFPR)
Campus Toledo / SAET 2026

13 a 16 de outubro de 2026



Sumário

Visão geral do minicurso

Visão Geral

Atividades Práticas
- Manipulação de Contêineres e Imagens
- Instalação do Kubernetes

Requisitos e Objetivo do Minicurso

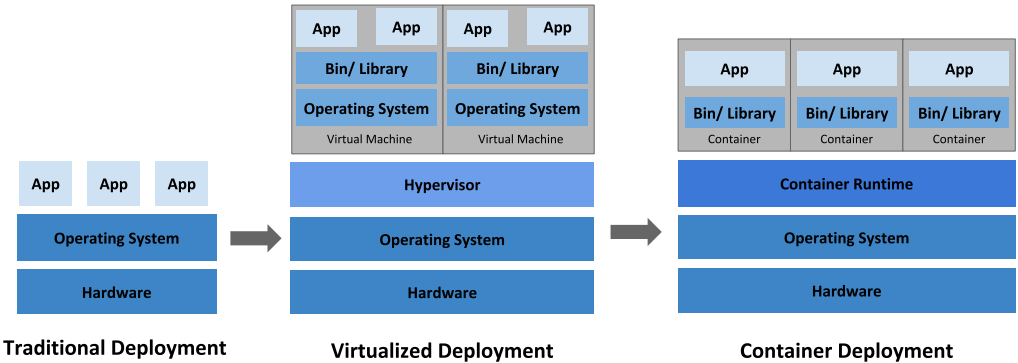
- **Requisitos:**
  - Conhecimentos básicos de linha de comando Linux, conceitos fundamentais de contêineres (Docker) e conhecimentos básicos de redes.
- **Objetivo:**
  - Capacitar os participantes a orquestrar aplicações em contêineres utilizando o Kubernetes e a automatizar o empacotamento, padronização e ciclo de vida das implantações por meio de Helm Charts.

Visão Geral

- **Kubernetes** é uma plataforma portável, extensível, e de código aberto para o gerenciamento de cargas de trabalho e serviços distribuídos em contêineres, que facilita tanto a configuração declarativa quanto a automação.
  - Serviços, suporte, e ferramentas para Kubernetes estão amplamente disponíveis.
- O Google tornou o Kubernetes um projeto de código-aberto em 2014. O Kubernetes combina mais de 15 anos de experiência do Google executando cargas de trabalho produtivas em escala, com as melhores ideias e práticas da comunidade.
- O nome **Kubernetes** tem origem no Grego, significando timoneiro (*helmsman*) ou piloto. **K8s** é a abreviação derivada pela troca das oito letras “ubernete” por “8”, se tornado K“8”s.

Fonte: `https://kubernetes.io/docs/concepts/overview/`

Evolução dos Contêineres



Fonte: `https://kubernetes.io/docs/concepts/overview/`

Implantação Tradicional

- Organizações executavam aplicações em servidores físicos
- Não havia como definir limites de recursos para aplicações em um mesmo servidor físico
  - Causava problemas de alocação de recursos
  - Algumas aplicações poderiam consumir quase todos os recursos de um servidor Deixando as outras com poucos recursos.

Implantação Tradicional

- Organizações executavam aplicações em servidores físicos
- Não havia como definir limites de recursos para aplicações em um mesmo servidor físico
  - Causava problemas de alocação de recursos
  - Algumas aplicações poderiam consumir quase todos os recursos de um servidor Deixando as outras com poucos recursos.
- **Solução:** executar cada aplicação em um servidor físico diferente
- **Consequência:** aumento de custos para as organizações.

Implantação Virtualizada

- Permite executar várias máquinas virtuais (VMs) em um servidor físico
- Cada VM é uma **máquina completa** que executa todos os componentes Sistema operacional e o hardware virtualizado
- Permite que as **aplicações** sejam **isoladas** entre as VMs
- Fornece um nível de segurança maior
  - **Informações** de uma aplicação **não podem ser acessadas livremente** por outras aplicações

Implantação em Contêineres

- Contêineres são semelhantes às VMs, mas têm **propriedades de isolamento** e **compartilham o kernel** do sistema operacional hospedeiro
  - Portanto são considerados **mais leves** do que as VMs
- Semelhante às VMs, um contêiner tem seu **próprio sistema de arquivos compartilhamento de CPU, memória, espaço de processo** e etc.
- São separados da infraestrutura subjacente
  - Por isso são portáveis entre nuvens e distribuições de sistema operacional

Motivos da popularidade dos contêineres

1. Criação e implantação ágil de aplicações
2. Desenvolvimento, integração e implantação contínuos
3. Separação de interesses entre Desenvolvimento e Operações (DevOps)
   - Imagens criadas na construção (build) do software em vez na implantação
4. A capacidade de observação (monitoramento)
5. Consistência entre desenvolvimento, teste e produção
6. Portabilidade de distribuição de nuvem e sistema operacional
7. Gerenciamento centrado em aplicações
8. Microserviços fracamente acoplados, distribuídos, elásticos e livres
9. Isolamento de recursos: desempenho previsível de aplicações
10. Utilização de recursos: alta eficiência e densidade

Porque usar Kubernetes e o que ele oferece?

1. Descoberta de serviço e balanceamento de carga
2. Orquestração de armazenamento
3. Lançamentos e reversões automatizadas de software
4. Empacotamento binário automático
   - Você fornece ao Kubernetes um cluster de nós que pode ser usado para executar tarefas nos contêineres
   - Você informa ao Kubernetes quanto de CPU e memória (RAM) cada contêiner precisa
   - O Kubernetes pode encaixar contêineres em seus nós para fazer o melhor uso de seus recursos
5. Autocorreção
   - O Kubernetes reinicia os contêineres que falham, substitui os contêineres, elimina os contêineres que não respondem
6. Gerenciamento de configuração e de segredos: informações confidenciais, senhas, tokens OAuth, chaves SSH, etc.

O que o Kubernetes não é ou não faz!

- O Kubernetes não é um sistema PaaS (plataforma como serviço) tradicional e completo
- Não opera no nível de hardware
- Não limita os tipos de aplicações suportadas
- Não implanta código-fonte e não constrói sua aplicação
- Não fornece serviços em nível de aplicação, tais como mensageria, banco de dados, cache, etc.
- Não dita soluções de logging, monitoramento ou alerta
  - Fornece algumas integrações como prova de conceito e mecanismos para coletar e exportar métricas
- Não fornece nem adota sistemas abrangentes de configuração de máquinas, manutenção, gerenciamento ou autocorreção

Kubernetes não é um simples sistema de orquestração

- A definição técnica de **orquestração** é a execução de um fluxo de trabalho definido: primeiro faça A, depois B e depois C.
- Na verdade, o **Kubernetes elimina** a necessidade de orquestração:
  - Em contraste, ele compreende um conjunto de processos de controle independentes e combináveis
  - Estes processos conduzem continuamente o estado atual em direção a um estado desejado
  - Não importa como você vai de A para C
- O controle centralizado também não é necessário
  - Resulta em um sistema que é mais fácil de usar e mais poderoso, robusto, resiliente e extensível

Gerencia aplicações e sua infraestrutura.

Não gerencia máquinas!

Manipulação de Contêineres com Docker

- Instalação do Docker
  - `https://docs.docker.com/engine/install/ubuntu/`

1. Baixar e executar um contêiner Nginx mapeando a porta 8080 do host para a 80 do contêiner
   ```
   $ docker run -d --name webserver -p 8080:80 nginx:alpine
   ```
2. Listar contêineres em execução
   ```
   $ docker ps
   ```
3. Inspecionar os logs do contêiner
   ```
   $ docker logs webserver
   ```
4. Executar um comando interativo dentro do contêiner
   ```
   $ docker exec -it webserver sh
   ```
5. Parar e remover o contêiner
   ```
   $ docker stop webserver
   $ docker rm webserver
   ```

Construção de uma Aplicação com Dockerfile I

Vamos criar uma aplicação Web simples utilizando `Node.js` e empacotá-la em uma imagem Docker customizada.

1. Crie o código da aplicação (`app.js`)

```
const express = require('express');
const app = express();
const PORT = 3000;

app.get('/', (req, res) => {
  res.send('<h1>Minicurso K8s & Helm: Aplicação rodando no
      Docker!</h1>');
});

app.listen(PORT, () => {
  console.log(`Servidor rodando na porta ${PORT}`);
});
```

Construção de uma Aplicação com Dockerfile II

2. Crie o arquivo de dependências (`package.json`):

```
{
  "name": "app-minicurso",
  "version": "1.0.0",
  "main": "app.js",
  "dependencies": {
    "express": "^4.18.2"
  }
}
```

Construção de uma Aplicação com Dockerfile III

3. Crie um arquivo chamado `Dockerfile` no mesmo diretório:

```
# Usa uma imagem oficial e leve do Node.js
FROM node:18-alpine

# Define o diretório de trabalho dentro do contêiner
WORKDIR /usr/src/app

# Copia os arquivos de definição de dependência
COPY package*.json ./

# Instala as dependências dentro da imagem
RUN npm install

# Copia o restante do código-fonte
COPY . .

# Expõe a porta em que a aplicação escuta
EXPOSE 3000

# Comando para iniciar a aplicação quando o contêiner subir
CMD ["node", "app.js"]
```

Construção de uma Aplicação com Dockerfile IV

4. Construa a imagem com a tag `app-minicurso:1.0`
   ```
   $ docker build -t app-minicurso:1.0 .
   ```
5. Execute a aplicação empacotada
   ```
   $ docker run -d -p 3000:3000 --name app-container app-minicurso:1.0
   ```
6. Teste via cURL ou no navegador (`http://localhost:3000`)
   ```
   $ curl http://localhost:3000
   ```

Instalação e Ferramentas do Kubernetes

- Instalação do Kubernetes
  - Ambientes de Produção: `https://kubernetes.io/docs/setup/`
  - Ambientes Dev: `https://canonical.com/microk8s`
- Ferramentas: kubeadm, kubectl, etc.
  - `https://kubernetes.io/docs/tasks/tools/`
- Distribuições empacotadas: kind, minikube, microk8s

Continua...

Perguntas e Discussão

Prof. Alexandre Huff
alexandrehuff@utfpr.edu.br
Universidade Tecnológica Federal do Paraná